Inventory Import Review - detailed recording guide (DRAFT)

The user has confirmed idea approval. This recording guide remains provisional because the actual input files have not yet been supplied. Use it only after the approved text is checked, the actual permitted inputs are available, and this guide is reconciled with those files. Proposed control/function names are explicit. Every code step below has a mandatory line-number field; the actual files must supply those numbers before recording. Keep this guide and all project materials off the recorded screen.

The SAY text is a cue for natural synchronized speech, not a requirement to recite mechanically. Every meaningful click, selection, scroll, edit, menu choice and app switch must name its app, exact target, purpose and observed result. If you need an extra action, narrate that actual action too. Never say a test passed because this guide predicts it. The expected checks below must be observed.

SAY	Green: speak just before and during the paired action. Use the cursor to point to the target as you name it.
ACTION	Blue: perform the stated visible operation. Pause about a second after clicks and let dialogs finish.
EDIT	Purple: enter only the specified code, one short segment at a time. Keep each continuous typing/mouse-idle interval under 45 seconds.
CHECK	Gold: compare the actual visible result with the stated expectation. This is not proof of a pass until you see it.
IF WRONG	Red: name the actual app, field or line, expected result, observed problem, correction and retest. Do not invent an error. Occasional corrected slips are allowed; restart if searching or repeated mistakes dominate.

Mandatory location and line-number standard

Every spoken instruction must be understandable to a first-time listener without relying on "here", "this", "next line" or an unnamed scroll. For source code, state Visual Studio Code, exact file, actual current line number, nearby code anchor, precise edit and reason. For page controls, state Firefox, visible control label, position relative to another visible control, action, purpose and result location. Announce the active window, scroll direction and destination before scrolling; point to the target before acting.

Line tokens such as {{S0.VALIDATOR_STUB}} are unresolved authoring fields, NOT numbers to read aloud. They must be replaced with verified numeric lines from the actual input and staged output before this guide becomes recording-ready. Source line numbers come from VS Code's gutter, not from wrapped visual rows. S0 is the unchanged starter and its initial Save As copy; S1 is the output after inserting the validator; S2 is the saved output after adding preview clearing. Later edits shift later lines. Use the matching state, never an old starter number for the changed file. See Location_And_Line_Map_DRAFT.md.

CHECK	Finalization must resolve every line token, check every quoted code anchor against the actual file and verify the browser's actual control layout. Keep line-number display on and keep formatting stable. Do not run Format Document or an unplanned format-on-save operation after mapping; if formatting changes physical lines, regenerate and verify the map before recording. If a real on-camera correction changes the numbering, read and announce the current gutter number and anchor before the next affected instruction.
CHECK	The proposed page layout is: title, local-only note, Inventory rows label/text area, Review import button directly below it, live status directly below the button, error list below the status, then Preview heading/table. The table columns are Item ID on the left, Location in the middle and Quantity on the right. Actual inputs must confirm this order; revise locators if their layout differs.

Reviewer feedback carried into this guide

CHECK	Use short, clearly pronounced sentences at normal playback speed. Before an action, state what you will do, the exact app/control and its location, and why. Speak during meaningful interaction and confirm the visible result afterward. Do not replace synchronized narration with a description after several silent actions. Brief natural thinking pauses are fine; repeated unexplained field changes are not.
CHECK	Say left-click for each primary mouse action; name right-clicks, double-clicks, shortcuts and left-button drags when used. Repeat the field name and location on every test. Say the actual filename or dialog name instead of "this file" or "this pop-up". Browser testing must be as specific as coding.
CHECK	Before deleting code or a TODO/comment, announce the actual current line and selected text. The proposed starter has no TODO markers, so do not invent a TODO-removal step. If the actual source differs, reconcile the guide first. Preserve adjacent statements and recheck line numbers after any deletion.
CHECK	When an error appears, immediately name the observed error and its app/file/control location. Explain the established cause, or say which cause you are checking. State the intended correction, do it visibly, then retest and describe the observed result. Never silently fix it or read a fabricated error from a script.
IF WRONG	Conditional error-narration pattern, not a factual claim: "In [actual app and file/control], at [verified line or visible location], I expected [expected result] but observed [actual error]. I am checking [specific suspected cause]. [State the cause only after confirming it.] I will [exact correction] because [reason], then rerun [named case] and check [result location]." Replace every bracket with the real incident or omit this pattern if no error occurred.

0. Readiness and setup - OFF SCREEN

CHECK	The user confirmed idea approval. Confirm the Goal and apps match the actual Studio approval and continue in that same task. If the reviewer changes the Goal, app list, scope or materials, synchronize the proposal, opening document, TaskRecorder and guide before recording.
CHECK	Actual input 1.html and input 2.md were prepared through the permitted process and checked. The starter has parseSource, renderRows and reviewImport scaffolding; validateBatch returns an empty array; the review handler omits clearing rows. The source text, IDs and messages match this guide. No completed output exists in the recording folder.
ACTION	Use a fresh neutral folder such as /Users/Shared/StockReview_Record containing only the two actual inputs. Rehearse in a separate disposable folder; do not move a completed rehearsal output into the fresh recording folder. Keep both original inputs unchanged.
ACTION	Prepare a Google Docs document in Firefox containing only the exact approved Goal, with no heading, approval ID, project instructions, or additional proposal text. Use the allowed account setup and hide personal account names/photos. Keep only this opening document behind TaskRecorder. Do not use Notes or TextEdit for the opening.

ACTION	In Visual Studio Code, prepare the neutral folder with Explorer visible, readable text and word wrap for the brief. Prepare Firefox. Disable substantive AI assistance and visible assistant panels. Do not display Codex, Gemini, GitHub, Studio, these scripts or the odin repository path during the take.
ACTION	Before the take, record a short microphone sample and listen back at normal playback speed. Use a quiet room, close room windows where practical, reduce fan/hum and traffic/voice noise without disabling required equipment, and use a headset microphone if available. Speak distinctly without rushing. If audio is unclear, correct the setup and retest before the full recording; do not plan to edit audio afterward.
ACTION	Before recording, hide Firefox's bookmarks toolbar, close unrelated tabs and inspect the address-bar suggestions, browser profile and the Goal document's account corner. Confirm no project links, login bookmarks or private details remain visible at the recording resolution. Check again in a short rehearsal sample.
ACTION	Quit unused apps and remove unused pinned Dock icons before recording, including messaging, mail, map and calling apps when not needed. Keep TaskRecorder and Accessibility Inspector running and preserve required background/security services. Keep the side Dock visible and do not try to hide it during the take.
CHECK	TaskRecorder 2.4.4: all required permissions green, microphone on and tested, correct screen selected, Task Description replaced with only the exact approved Goal, Used Apps matching approved Visual Studio Code and Firefox. Google Docs is opening-only, not extra substantive task work.
CHECK	Accessibility Inspector remains running with its window closed; the Dock icon indicator stays visible. Side Dock does not auto-hide, default cursor, visible menu-bar seconds, required display resolution, power and sufficient storage, notifications/PII hidden. Apply one permitted appearance change before recording. Keep the recording in a native macOS environment.
CHECK	Use ordinary windows, with the menu bar and Dock visible. Do not use full-screen or full-screen Split View. Rehearse normal window resizing and Firefox text selection before the actual take.
CHECK	Plan roughly 30-42 minutes including intro and epilogue; measure actual rehearsal rather than padding. Writer total range is 15:00-50:59. Coding typing stays within 30% of total time. No continuous typing/mouse inactivity over 45 seconds except in the epilogue. If the natural task cannot meet these limits, revise it before recording.

1. Brief Goal opening - under 20 seconds

ACTION	In TaskRecorder, left-click Start Recording. When recording starts, left-click its minimize control so only the Goal document in Firefox remains visible.
SAY	"I will be beginning my task soon. Here is the workflow I have in mind. I will inspect the inventory-import rules, implement batch validation in Visual Studio Code, and check the local preview in Firefox."
ACTION	Point briefly to the Goal paragraph. Close the opening document tab using its tab close button, then left-click Visual Studio Code in the side Dock. Announce this move as you do it. Do not read the full Goal aloud.

CHECK	The complete intro stays under 20 seconds. The visible Goal is exact and no approval ID, project page or private information appears.

2. Read the requirements - about 0:20-4:30

SAY	"In Visual Studio Code's Explorer pane on the far left, beneath the open folder name, I am left-clicking input 2.md, listed below input 1.html. I want the import rules and expected outcomes before changing the page."
ACTION	Open input 2.md, pause and point to its title, then the format and business-rule headings.
SAY	"In input 2.md at line {{B.FORMAT}}, below the document title in the format section, each nonblank record has item ID, location and quantity separated by a vertical bar. This is a small plain-text format, not a general CSV parser with quoted fields."
ACTION	Point to the header item_id|location|quantity, the three-field rule, whitespace handling and physical line-number requirement. If below view, say you are scrolling down within the input 2.md editor to those rules before scrolling.
SAY	"In input 2.md at line {{B.BUSINESS_RULES}}, below the format rules, item IDs and locations are required. IDs must be compared after trimming and uppercasing. Quantities use digits only, from zero through 9999. A blank string must not become a valid zero just because Number converts it to zero."
ACTION	Point individually to required values, duplicate normalization and quantity grammar/range. Keep the brief unchanged.
SAY	"In input 2.md at line {{B.BATCH_RULE}}, below the quantity rules, any error rejects the whole preview. A later review must clear old rows and errors, so a rejected batch cannot appear alongside an earlier successful preview."
ACTION	Point to all-or-nothing rejection and valid-invalid-valid recovery. Scroll explicitly to this heading if needed.
SAY	"I am scrolling down in the input 2.md editor to the acceptance section beginning with I01 at line {{B.I01}} and ending with I12 at line {{B.I12}}. These groups cover normal input, zero and leading zeros, duplicates, missing fields, invalid quantities, structural errors, safe text, recovery and bounds. I will compare the displayed rows, totals and error lines with these expectations."
ACTION	Scroll to the acceptance groups, point to each group heading, and locate I09's literal Store value and I10's three-review sequence. Do not claim a result yet.

3. Inspect the starter and save the output copy - about 4:30-8:00

SAY	"In Visual Studio Code's Explorer pane on the far left, I am left-clicking input 1.html, listed above input 2.md, to inspect the starter. I will preserve this source and create a separate output before editing."

ACTION	Open input 1.html and point to its tab. Scroll in this HTML editor to the body controls, naming the destination before the scroll.
SAY	"In input 1.html, I am at the body controls beginning on line {{S0.HTML_CONTROLS_START}}, below the closing head tag. The source text area is at line {{S0.SOURCE}}, the Review import button at {{S0.BUTTON}}, the live status at {{S0.STATUS}}, the error list at {{S0.ERRORS}}, and the preview table body at {{S0.ROWS}}. These IDs connect the visible controls to the script below."
ACTION	Point to the five IDs and the local-review-only note. Do not edit the original.
SAY	"I am scrolling down inside the input 1.html editor to parseSource at line {{S0.PARSER_START}}, below the DOM element references. It already checks the header and three fields, trims values, tracks physical line numbers and skips blank lines. I will retain those parsing decisions."
ACTION	Point to the header comparison at {{S0.HEADER_CHECK}}, field-count check at {{S0.FIELD_COUNT_CHECK}} and record object at {{S0.RECORD_BUILD}} with number, id, location, rawQuantity and quantity. Announce each current line as you point to it. Check these actual property names against the guide.
SAY	"I am scrolling farther down inside input 1.html to validateBatch at line {{S0.VALIDATOR_START}}, below parseSource. Its return-empty-array statement on line {{S0.VALIDATOR_STUB}} does not reject invalid business values. Directly below the validator, renderRows begins at line {{S0.RENDERER_START}}; its cell text assignment at line {{S0.SAFE_CELL_TEXT}} will be preserved."
ACTION	Point to the empty validator, then document.createElement and td.textContent in the renderer. If you scroll between them, announce the function you are seeking.
SAY	"I am scrolling down inside input 1.html to reviewImport at line {{S0.HANDLER_START}}, below renderRows and near the end of the script. It clears errors and combines parser and validator errors before rendering, but it never clears previous preview rows. That explains how an old preview could remain after a rejected attempt."
ACTION	Point to errorsEl.replaceChildren at {{S0.CLEAR_ERRORS}}, the combined errors array at {{S0.COMBINE_ERRORS}}, rejection return at {{S0.REJECT_RETURN}} and renderRows call at {{S0.RENDER_CALL}}. Name each current line and its neighboring statement while discussing it. Do not claim you observed the bug in Firefox yet.
SAY	"In Visual Studio Code, I am left-clicking File in the top menu bar, then Save As in that menu. In the Save As dialog, I am selecting the filename field and entering final output 1.html in the neutral task folder. I will left-click Save in that dialog to preserve both original inputs."
ACTION	Left-click File, then Save As. In the dialog, select the neutral folder and filename field, enter final output 1.html, and left-click Save. Announce each selection; hide any private path before the take, rather than repairing it on camera.
CHECK	Active tab is final output 1.html, both inputs are separate and unchanged, and the working copy still contains the incomplete starter. If the name or location is wrong, correct Save As before editing.

4. Observe both baseline defects in Firefox - about 8:00-11:00

SAY	"I am left-clicking the Firefox icon in the side Dock, then File in the top menu bar and Open File in that menu. In the Open File dialog, I am selecting final output 1.html from the neutral folder and left-clicking Open. I want to observe the still-unmodified starter copy."
ACTION	Switch apps. Choose File > Open File, select the neutral task folder and final output 1.html, then left-click Open. Point to the local-only note and Review import button.
SAY	"In Firefox's Inventory rows text area below the local-only note and above the Review import button, I am entering a header and two rows whose IDs differ only in capitalization. Under the brief, they should count as duplicate A101 records."
EDIT	Enter these three lines in the source text area: item_id|location|quantity / A101|North store|3 / a101|Loading bay|2. Each slash here means a line break; do not type slashes.
SAY	"I am left-clicking Review import directly below the Inventory rows text area. I expect this incomplete validator to accept the duplicate batch; I will describe the result that actually appears."
ACTION	Left-click the button, pause and point to the status directly below the button and the two rows in the Preview table farther down the page.
CHECK	Intended baseline: two rows displayed, no duplicate error, and a Validated status. The row IDs remain A101 and a101. If this does not happen, stop and reconcile the actual starter with the guide before continuing; do not invent the expected baseline.
SAY	"In that Inventory rows text area above Review import, on its first input line, I am replacing only the header's item_id with id, leaving the two data rows. I am left-clicking Review import again to observe the stale-preview defect."
ACTION	Left-click the first header field, select only item_id and type id. Left-click Review import, pause, and point first to the header error in the list below the status, then to the old rows in the Preview table beneath that list.
CHECK	Intended baseline: the header is rejected, but the earlier two preview rows remain. An old preview is not evidence of a successful current review. State the actual observed result.
SAY	"I am returning to Visual Studio Code through the side Dock. The observed issues are missing business validation and preview rows that survive a later rejection. I will fix them in the separate output file."
ACTION	Left-click Visual Studio Code, select final output 1.html and confirm the tab name before any edit.

5. Implement the batch validator - about 11:00-17:00

SAY	"In Visual Studio Code's final output 1.html tab at the top of the editor, I am scrolling upward to validateBatch on line {{S0.VALIDATOR_START}}, above renderRows. On line {{S0.VALIDATOR_STUB}}, directly below the function declaration, I am selecting only return []; and replacing that statement while leaving the outer braces intact."

ACTION	Scroll upward in the output editor to line {{S0.VALIDATOR_START}}. Point to the function name, then select only return []; on line {{S0.VALIDATOR_STUB}}, leaving that physical line available for replacement. Do not remove the function declaration or closing brace.
SAY	"On line {{S1.VALIDATOR_ERRORS}} in final output 1.html, directly below the validateBatch declaration, I am creating one error list and one Set of normalized IDs. The Set will detect duplicate IDs in this review only."
EDIT	In final output 1.html at line {{S1.VALIDATOR_ERRORS}}, directly below the function declaration, type: `const errors = [], seen = new Set();` Then pause and point to both names.
SAY	"On line {{S1.RECORD_LOOP}} in final output 1.html, directly below the errors-and-seen declaration, I am looping over every parsed record. I will collect all business issues rather than stopping after the first bad value."
EDIT	In final output 1.html at line {{S1.RECORD_LOOP}}, directly below the declarations, type: `records.forEach(record => {` Place the cursor inside that callback, preserving or managing the editor's auto-inserted closing characters so the final code has one matching `});`.
CHECK	The outer function still has its own closing brace. Confirm actual auto-closing behavior from rehearsal; never type duplicate closing braces blindly.
SAY	"On line {{S1.NORMALIZE_ID}} in final output 1.html, inside the records.forEach callback and directly below its opening line, I am uppercasing the item ID before comparing it. The parser already trimmed the field. On line {{S1.ERROR_PREFIX}}, immediately below that assignment, I am creating the error prefix from the record's physical source line number."
EDIT	In final output 1.html, type `record.id = record.id.toUpperCase();` at line {{S1.NORMALIZE_ID}}, then `const prefix = 'Line ' + record.number + ': ';` at line {{S1.ERROR_PREFIX}} immediately below it. Pause and point to the uppercasing and number property.
SAY	"On line {{S1.REQUIRED_ID}} in final output 1.html, directly below the prefix declaration inside the callback, I am rejecting an empty item ID. I will still inspect the other fields on that row."
EDIT	In final output 1.html at line {{S1.REQUIRED_ID}}, below the prefix declaration, type: `if (!record.id) errors.push(prefix + 'item ID is required.');`
SAY	"On line {{S1.DUPLICATE_IF}} in final output 1.html, directly below the required-ID check, I am checking the normalized ID against seen. Its error is added on line {{S1.DUPLICATE_ERROR}}, inside that if block. I only test a nonempty ID, so an empty value is reported as missing rather than treated as a duplicate key."
EDIT	In final output 1.html, type `if (record.id && seen.has(record.id)) {` at line {{S1.DUPLICATE_IF}}; type `errors.push(prefix + 'duplicate item ID ' + record.id + '.');` at {{S1.DUPLICATE_ERROR}} inside that block; close only this if block with `}` at {{S1.DUPLICATE_END}}. Pause after each short entry.

SAY	"On line {{S1.SEEN_ADD}} in final output 1.html, immediately below the duplicate-check closing brace on line {{S1.DUPLICATE_END}}, I am adding a nonempty ID to seen. On line {{S1.REQUIRED_LOCATION}}, directly below that addition, I am rejecting an empty location. This keeps ID identity separate from the quantity and location checks."
EDIT	In final output 1.html, type `if (record.id) seen.add(record.id);` at line {{S1.SEEN_ADD}} below the duplicate block, then `if (!record.location) errors.push(prefix + 'location is required.');` at {{S1.REQUIRED_LOCATION}} directly below it. Pause and point to each condition.
SAY	"On line {{S1.QUANTITY_IF}} in final output 1.html, directly below the location check, I am validating the raw quantity string with a digits-only expression and the parsed number against 9999. The error message belongs on line {{S1.QUANTITY_ERROR}}, inside that if block. This rejects blanks, signs, decimals and exponent syntax while allowing zero and leading zeros."
EDIT	In final output 1.html, type `if (!/^\d+$/.test(record.rawQuantity) || record.quantity > 9999) {` at line {{S1.QUANTITY_IF}} below the location check. Type `errors.push(prefix + 'quantity must be an integer from 0 to 9999.');` at {{S1.QUANTITY_ERROR}} inside this block, and close it with `}` at {{S1.QUANTITY_END}}. Pause and inspect the backslash-d, plus sign, rawQuantity property, and strict greater-than boundary.
SAY	"In final output 1.html, I am closing the quantity check on line {{S1.QUANTITY_END}} and the per-record callback on line {{S1.RECORD_LOOP_END}}. On line {{S1.RETURN_ERRORS}}, below that callback and above the function closing brace at {{S1.VALIDATOR_END}}, I am returning the collected errors from validateBatch. The existing review handler can now reject the entire batch before any preview is rendered."
EDIT	In final output 1.html, close the record callback with `});` on line {{S1.RECORD_LOOP_END}}, below the quantity-check closing brace. Type `return errors;` at {{S1.RETURN_ERRORS}} below that callback and above the function closing brace at {{S1.VALIDATOR_END}}. Leave the original function-closing brace intact.
CHECK	The complete validator matches the block below. This block is an offscreen reference; do not paste a completed solution from this guide during recording. Enter the short segments through the recorded work and understand each one.

```
function validateBatch(records) {
  const errors = [], seen = new Set();
  records.forEach(record => {
    record.id = record.id.toUpperCase();
    const prefix = 'Line ' + record.number + ': ';
    if (!record.id) errors.push(prefix + 'item ID is required.');
    if (record.id && seen.has(record.id)) {
      errors.push(prefix + 'duplicate item ID ' + record.id + '.');
    }
    if (record.id) seen.add(record.id);
    if (!record.location) errors.push(prefix + 'location is required.');
    if (!/^\d+$/.test(record.rawQuantity) || record.quantity > 9999) {
      errors.push(prefix + 'quantity must be an integer from 0 to 9999.');
    }
  });
  return errors;
}
```

IF WRONG	If there is a syntax underline, inspect the actual function in the output editor: callback closing `});`, nested if braces, quotes and property spelling. Name any real correction, make it on camera and verify it. Do not move on with an unexplained syntax error.

6. Clear stale rows, review and save - about 17:00-19:00

SAY	"I am scrolling down in Visual Studio Code's final output 1.html to reviewImport, now at line {{S1.HANDLER_START}} below renderRows. Directly below its opening brace and above the existing errorsEl.replaceChildren statement at line {{S1.CLEAR_ERRORS}}, I am inserting a new line, which will be line {{S2.CLEAR_ROWS}}, containing rowsEl.replaceChildren(). Every review will begin with an empty preview."
ACTION	Scroll downward to line {{S1.HANDLER_START}} in the output editor. Point to its opening brace, then to errorsEl.replaceChildren on current line {{S1.CLEAR_ERRORS}}. Insert the new clearing statement immediately above it. After insertion, confirm the new rows-clearing line is {{S2.CLEAR_ROWS}} and errors-clearing line is {{S2.CLEAR_ERRORS}}.
EDIT	In final output 1.html at new line {{S2.CLEAR_ROWS}}, directly below the reviewImport declaration and above errorsEl.replaceChildren, type exactly: `rowsEl.replaceChildren();`
CHECK	The handler now clears rows and errors on every click, then parses, validates, rejects on any error, and renders only after the error check. Do not put the new line inside the success branch; that would leave stale rows after failure.
SAY	"I am scrolling upward inside final output 1.html to renderRows at current line {{S2.RENDERER_START}}, above reviewImport. I am keeping the td.textContent assignment on line {{S2.SAFE_CELL_TEXT}}, below the new table-cell declaration. I will then scroll downward to the success status at {{S2.SUCCESS_STATUS}}, below the total calculation, and confirm the local-review-only wording."
ACTION	Point to the safe cell assignment and the success/rejection messages. Scroll deliberately between these named regions if needed. Check no network or storage code was added.
SAY	"In Visual Studio Code, I am left-clicking File in the top menu bar, then Save in that menu, to save final output 1.html. I will check the output tab at the top of the editor for the absence of an unsaved indicator, then test this exact saved file in Firefox."
ACTION	Left-click File > Save, pause and check the tab's unsaved indicator is gone. If using Command-S, announce the shortcut before pressing it.

7. Set up readable source-and-browser verification - about 19:00-20:30

SAY	"I am opening input 2.md in Visual Studio Code and arranging it on the left, with Firefox's output page on the right. This lets me copy each supplied test and compare its expected result without showing my offscreen guide."
ACTION	Left-click input 2.md in Explorer. Using ordinary window title bars and edges, move and resize Visual Studio Code to the left and Firefox to the right. Before each drag, name the window and edge you are moving. Keep both readable, avoid substantial overlap, and keep the menu bar and Dock visible. Do not enter full-screen Split View. If the screen cannot support readable side-by-side windows, use one ordinary active window at a time and narrate each switch.
SAY	"In Firefox on the right, I am reloading final output 1.html from the saved local file. The text area may retain old input; I will replace it with I01 before reviewing."

ACTION	Left-click Firefox, use its reload button, pause and point to the local-only note. If pressing Command-R instead, announce that shortcut.
CHECK	The table is the page's actual output, not a browser source tab. Both input files remain unchanged. Each new case below replaces the full text-area contents before clicking Review import.

8. Complete each acceptance group - about 20:30-32:00

ACTION	For each group, the selection is data only from the actual input 2.md. Do not select a Markdown fence, case title or Expected paragraph. Use visible Edit menus for Copy, Select All and Paste where practical; if using Command-C, Command-A or Command-V, name the shortcut when you use it. A clipboard transfer of supplied input data is not a transfer of completed solution code.
CHECK	Standard observation after every review: status, error list, preview row count, displayed IDs/locations/quantities and total where successful. A rejection must leave zero preview rows. Scroll in Firefox to the error list/table if necessary, naming which area you are bringing into view. Do not silently skip below-the-fold content.

8.01 I01 - Normalize a valid batch

SAY	"I am checking whether trimming and uppercasing produce two canonical item IDs and whether the quantity total is five."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling up to I01 at line {{B.I01}}, normalize a valid batch. I am holding the left mouse button to select its supplied input from line {{B.I01_DATA_START}} through line {{B.I01_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll up within it to the I01 heading at line {{B.I01}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I01_DATA_START}} through the last input line at {{B.I01_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I01's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	Two preview rows: A101 / North store / 3 and B202 / Loading bay / 2. Status: Validated 2 rows (5 total units). Local review only; nothing imported. No errors.

IF WRONG	If I01 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I01. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I01, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.02 I02 - Zero and leading-zero quantities

SAY	"I am checking that zero is accepted as a real quantity and that 003 displays as three, rather than being rejected or preserved as misleading text."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I02 at line {{B.I02}}, zero and leading-zero quantities. I am holding the left mouse button to select its supplied input from line {{B.I02_DATA_START}} through line {{B.I02_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I02 heading at line {{B.I02}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I02_DATA_START}} through the last input line at {{B.I02_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I02's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	Two rows, quantities 0 and 3; total 3. Zero is valid. Leading zeros normalize to numeric display.
IF WRONG	If I02 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I02. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I02, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.03 I03 - Duplicate IDs after trimming and uppercasing

SAY	"I am checking that A101 and the trimmed lowercase a101 are treated as the same ID, so the entire duplicate batch is rejected."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I03 at line {{B.I03}}, duplicate ids after trimming and uppercasing. I am holding the left mouse button to select its supplied input from line {{B.I03_DATA_START}} through line {{B.I03_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I03 heading at line {{B.I03}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I03_DATA_START}} through the last input line at {{B.I03_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I03's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	No preview rows. One error: Line 3: duplicate item ID A101.
IF WRONG	If I03 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I03. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I03, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.04 I04 - Missing location and missing ID

SAY	"I am checking both missing-location and missing-ID errors, including their original source line numbers. Neither row should appear as an accepted preview."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I04 at line {{B.I04}}, missing location and missing id. I am holding the left mouse button to select its supplied input from line {{B.I04_DATA_START}} through line {{B.I04_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."

ACTION	Left-click the brief editor, scroll down within it to the I04 heading at line {{B.I04}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I04_DATA_START}} through the last input line at {{B.I04_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I04's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	No preview rows. Two errors: Line 2: location is required. / Line 3: item ID is required.
IF WRONG	If I04 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I04. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I04, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.05 I05 - Blank, fractional, negative and exponent quantities

SAY	"I am checking that raw quantity grammar rejects a blank, a decimal, a negative value and exponent syntax, even when Number could convert some values."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I05 at line {{B.I05}}, blank, fractional, negative and exponent quantities. I am holding the left mouse button to select its supplied input from line {{B.I05_DATA_START}} through line {{B.I05_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I05 heading at line {{B.I05}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I05_DATA_START}} through the last input line at {{B.I05_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I05's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."

ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	No preview rows. Four quantity errors on physical lines 2, 3, 4 and 5. Each says quantity must be an integer from 0 to 9999.
SAY	"In Firefox's error list below the status message and above the Preview heading, I am checking all four source line numbers, two through five. Blank, one-point-five, negative one and one-e-two are all invalid quantity strings, even if Number could convert some of them."
ACTION	Point to each error. If the list is below view, announce a downward scroll in the Firefox page to the quantity errors, then scroll. Confirm that the preview table is empty.
IF WRONG	If I05 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I05. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I05, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.06 I06 - Wrong number of fields

SAY	"I am checking that an extra pipe-separated field is rejected instead of silently dropping data."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I06 at line {{B.I06}}, wrong number of fields. I am holding the left mouse button to select its supplied input from line {{B.I06_DATA_START}} through line {{B.I06_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I06 heading at line {{B.I06}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I06_DATA_START}} through the last input line at {{B.I06_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I06's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."

ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	No preview rows. One error: Line 2: expected 3 fields.
IF WRONG	If I06 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I06. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I06, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.07 I07 - Wrong header

SAY	"I am checking that an unrecognized header is rejected rather than treating incorrectly labeled columns as inventory fields."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I07 at line {{B.I07}}, wrong header. I am holding the left mouse button to select its supplied input from line {{B.I07_DATA_START}} through line {{B.I07_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I07 heading at line {{B.I07}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I07_DATA_START}} through the last input line at {{B.I07_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I07's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	No preview rows. One error: Use header: item_id|location|quantity.

IF WRONG	If I07 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I07. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I07, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.08 I08 - Header without data rows

SAY	"I am checking that a header alone does not produce a successful empty-batch review."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I08 at line {{B.I08}}, header without data rows. I am holding the left mouse button to select its supplied input from line {{B.I08_DATA_START}} through line {{B.I08_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I08 heading at line {{B.I08}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I08_DATA_START}} through the last input line at {{B.I08_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I08's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	No preview rows. One error: Enter at least one data row.
IF WRONG	If I08 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I08. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I08, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.09 I09 - Markup-like location is literal text

SAY	"I am checking that markup-like text is displayed literally, using the safe renderer that already exists in the starter."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I09 at line {{B.I09}}, markup-like location is literal text. I am holding the left mouse button to select its supplied input from line {{B.I09_DATA_START}} through line {{B.I09_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I09 heading at line {{B.I09}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I09_DATA_START}} through the last input line at {{B.I09_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I09's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	One row, total 2; location displays the literal characters Store, rather than bold Store. No markup is executed.
SAY	"In the Firefox Preview table below the error list, in the middle Location column between Item ID and Quantity, I am checking that the angle brackets and b tags are visible characters. The word Store must not become formatted HTML."
ACTION	Point to the complete literal Store cell. Do not open an inspector or claim broader security testing. This check verifies the provided literal-text case.
IF WRONG	If I09 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I09. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I09, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.10 I10 - Valid-invalid-valid recovery

SAY	"In the input 2.md editor on the left, I am scrolling down to I10 at line {{B.I10}}, directly below the literal-markup test I09. I am reading its valid-invalid-valid sequence before returning to the earlier input blocks."
ACTION	Point to the I10 heading at {{B.I10}} and its three-stage expected result, then scroll upward to I01 for the first stage.
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling upward to I01 at line {{B.I01}} for the first part of I10's recovery test. I will run a valid review, then I03's invalid duplicate review, then I01 again. For each stage I will copy the selected source through Visual Studio Code's Edit menu, left-click Inventory rows in Firefox below the local-only note and above Review import, announce Command-A and Command-V to replace the full field contents, and left-click Review import directly beneath it."
ACTION	Scroll to I01 in the brief. Select only its three-line data block, choose Edit > Copy, left-click Firefox's source text area, select all text there, paste and left-click Review import. Narrate each app focus, selection, copy, replacement and button left-click.
CHECK	First stage: exactly two rows, total five and no errors.
SAY	"I am scrolling downward in the left-hand brief to I03 at line {{B.I03}}, copying its duplicate-ID input through Edit and Copy. In Firefox, I am left-clicking Inventory rows below the local-only note and above Review import, pressing Command-A and Command-V to replace that field's full contents, then left-clicking Review import directly below it. I am checking that no old successful preview remains."
ACTION	Scroll to I03, select its data only, copy, left-click the Firefox textarea, select all, paste and left-click Review import. Pause and point to the rejection and empty preview.
CHECK	Middle stage: exactly one duplicate error and no preview rows.
SAY	"I am scrolling upward in the brief to I01 at line {{B.I01}} and copying the valid input through Edit and Copy. In Firefox, I am left-clicking Inventory rows below the local-only note and above Review import, pressing Command-A and Command-V to replace the whole field, then left-clicking Review import directly below it. This must clear the error and render exactly two rows rather than accumulate old rows."
ACTION	Repeat the explicit copy/replace/review steps using I01. Point to both row IDs and the cleared error area.
CHECK	Final stage: two rows, total five, no errors. If any stage differs, this sequence has not passed.
IF WRONG	If I10 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I10. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.

SAY	After observing I10, describe each actual stage: successful two-row preview, duplicate rejection with no preview, then successful two-row preview with cleared errors. State whether this verifies recovery; do not claim a stage you did not observe.

8.11 I11 - Blank lines, trimmed header and line endings

SAY	"I am checking that blank lines and surrounding whitespace do not create false errors or change normalized row counts and totals."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I11 at line {{B.I11}}, blank lines, trimmed header and line endings. I am holding the left mouse button to select its supplied input from line {{B.I11_DATA_START}} through line {{B.I11_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I11 heading at line {{B.I11}} and point to the expected result. Hold the left mouse button and drag-select from the leading blank data line at source line {{B.I11_DATA_START}} through the last input line at {{B.I11_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I11's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	Two normalized rows, total 5, with no error. Parser must also accept CRLF and an optional leading UTF-8 BOM. Verify those parser properties in source and in offscreen validation; do not claim a Firefox CRLF/BOM check unless the actual input preserves those characters.
CHECK	Preserve the leading and interior blank lines when selecting the actual I11 block. The on-camera copy/paste check verifies visible blank-line and trimming behavior. Only claim CRLF or BOM handling if you actually preserve and test those characters; otherwise verify the parser's split/BOM logic in source and keep that distinction in your reflection.
IF WRONG	If I11 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I11. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I11, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

8.12 I12 - Inclusive maximum and over-limit rejection

SAY	"I am checking the inclusive quantity maximum first, then the first value above that maximum."
SAY	"In Visual Studio Code's input 2.md on the left, I am scrolling down to I12 at line {{B.I12}}, inclusive maximum and over-limit rejection. I am holding the left mouse button to select its supplied input from line {{B.I12_DATA_START}} through line {{B.I12_DATA_END}}, excluding the Markdown fences, so I can compare the local result with the brief. With the input selected, I am left-clicking Edit in Visual Studio Code's top menu bar, then Copy in that menu."
ACTION	Left-click the brief editor, scroll down within it to the I12 heading at line {{B.I12}} and point to the expected result. Hold the left mouse button and drag-select from the header line at source line {{B.I12_DATA_START}} through the last input line at {{B.I12_DATA_END}}, excluding Markdown fences and explanations. Choose Edit > Copy. Keep the source unchanged.
SAY	"In Firefox on the right, I am left-clicking inside the Inventory rows text area below the local-only note and above Review import. I am pressing Command-A to select all text inside Inventory rows, then Command-V to paste I12's supplied text. I am left-clicking Review import directly below the Inventory rows text area to run a new local review."
ACTION	Left-click inside Inventory rows, the text area immediately above Review import; announce and press Command-A to select all its contents, then Command-V to paste the copied data; point to the header and changed values. Left-click Review import directly below Inventory rows, pause, and inspect the status directly below the button, the error list below that status and the Preview table beneath the error list.
CHECK	First, 9999 is accepted as one row and total 9999. Then replace 9999 with 10000 and review again: no preview, one quantity error on Line 2.
SAY	"The Firefox review accepted 9999, which is the allowed maximum. In Firefox's Inventory rows text area, below the local-only note and above Review import, on input line two after the second vertical bar, I am selecting only 9999 and replacing it with 10000 and left-clicking Review import again to check the first disallowed value."
ACTION	Select the quantity in the second source line, type 10000, left-click Review import, pause and point to the Line 2 quantity error and empty table.
CHECK	The maximum is inclusive: 9999 passes and 10000 rejects. The earlier maximum-value preview must disappear on rejection.
IF WRONG	If I12 differs from its written expectation, say the case ID, app, expected result and actual result. Return to final output 1.html in Visual Studio Code; inspect the implicated parser, validator, renderer or review handler; correct only an actual problem; save; reload the output in Firefox; and repeat I12. If validation or reset behavior changed, rerun I01, I03 and I10 as relevant. Do not claim an unseen pass.
SAY	After observing I12, state its actual status, relevant error lines or accepted rows, and whether the visible outcome matches the brief. Name the result location: status below Review import, errors beneath status, and Preview table below the error list. Do not read an expected result as though it occurred.

9. Final saved-file and usable-state review - about 32:00-34:00

SAY	"I am leaving the completed page in a valid local-review state by scrolling upward in the input 2.md brief to I01 at line {{B.I01}} and copying that supplied block through Edit and Copy. In Firefox, I am left-clicking Inventory rows below the local-only note and above Review import, pressing Command-A and Command-V to replace the field, then left-clicking Review import directly beneath it. I will check the normalized IDs, two rows, five total units and local-only wording."
ACTION	Copy I01 data from the unchanged brief, replace the Firefox textarea and left-click Review import. Pause and point to the exact successful status and both preview rows. Narrate the copy, app focus, replacement and left-click.
SAY	"I am returning to final output 1.html in Visual Studio Code. I am checking the filename and saved state, then validateBatch at current line {{S2.VALIDATOR_START}}, above renderRows, and the rows-clearing statement at {{S2.CLEAR_ROWS}}, directly below reviewImport. I will also point to the two separate original input files in Explorer before closing the workflow."
ACTION	Left-click Visual Studio Code, open the output tab and point to its name and absence of an unsaved indicator. Point to input 1.html and input 2.md in Explorer. Scroll to the completed validateBatch function at {{S2.VALIDATOR_START}}; then downward to the safe renderer assignment at {{S2.SAFE_CELL_TEXT}}, then downward to the start-of-review clearing line at {{S2.CLEAR_ROWS}}. Speak the current line number, file and nearby code before pointing to each region.
CHECK	The final code has no unresolved syntax error, unintended request/storage code or unseen changes. The two inputs remain unchanged. Keep the exact saved file from this recording. If you change the output after the final Firefox review, save and retest before stopping.

10. Closing reflection - target 3-4 minutes, maximum 4:00

CHECK	These are conditional speaking cues. Use only outcomes verified in the current take. If the task was not complete, state the limitation. The cursor may remain still during this epilogue. Do not extend it beyond four minutes. A real correction can replace part of the scripted reflection; do not append unlimited text or invent an error. Rehearse the reflection aloud at a clear natural pace, targeting 3-4 minutes and never more than four. Cover outcome, verification, approach, actual challenges, quality, limitations and improvement; do not shorten it to a one-minute summary or pad it with filler.
SAY	"The deliverable is final output 1.html, a standalone inventory-import review page. It previews a synthetic batch locally and makes no change to inventory or stored data. I preserved the original starter and requirements. The final file now validates business values before showing a preview, and it clears earlier results when a new review starts."
SAY	"I began with the requirements rather than changing the display first. The starter already knew how to split the plain-text format, preserve source line numbers and render cells as text. The missing work was deciding which records could be accepted together. I kept that business validation separate from parsing and rendering, so the different responsibilities remain easy to inspect."

SAY	"The main identity decision was to uppercase each already-trimmed item ID before comparing it with the Set of IDs seen in this batch. A101 and a101 refer to the same item under this brief. Checking the raw spelling first would miss that duplicate. I also validate the raw quantity text, because numeric conversion alone would accept an empty string as zero and exponent syntax as a number. Zero itself is valid, so it needs an explicit rule rather than a truthiness test."
SAY	"In the baseline, the duplicate batch was accepted and a later header error left that earlier preview on screen. I used those observed defects to check the purpose of the repair. In the completed page, a failure leaves no preview rows, and a valid-invalid-valid sequence removes old rows and errors instead of accumulating them. That keeps the displayed result tied to the latest review."
SAY	"I compared the acceptance groups in Firefox with the written expectations, including missing values, invalid quantities, header and field-count errors, zero, leading zeros and the upper boundary. I also checked that the markup-like location stayed literal text. I retained textContent rather than interpreting record values as HTML. My local browser observations apply to these test inputs; they do not establish that every possible file or browser has been tested."
SAY	"For a production importer, I would add real backend validation, authorization and a controlled import transaction, plus broader automated coverage and a documented file-size limit. Those features are outside this local review task. The principle here is that parsing a record is different from accepting a batch. Normalize before comparing identity, validate before rendering success, and keep any reported errors traceable to their original source lines."
IF WRONG	If a real correction occurred, replace part of the reflection with its case ID, observed issue, exact correction and successful retest. If a requirement remains unverified, state that limitation instead of claiming completion. Do not read the CRLF/BOM result as browser-tested unless it actually was.

11. Stop and archive

SAY	"The saved output and local checks are complete. I am left-clicking the TaskRecorder icon in the side Dock to bring its window forward, then its Stop Recording control to finish this continuous take. I will wait until archiving reaches Idle."
ACTION	Left-click TaskRecorder in the side Dock, then Stop Recording. Wait through Stopping and Archiving until Idle. Do not quit, rename, edit or interrupt the recording archive.

12. After recording - OFF SCREEN

CHECK	Open the matching video.mp4 in QuickTime and inspect beginning, middle and end for the right attempt, usable picture and clear speech. Check the archive contains the corresponding screenshots, video.mp4, frames.jsonl, ax.jsonl, input.jsonl, processes.jsonl and recording.log. Screenshot rate must exceed 0.5 per recording second. Do not mix attempts.
ACTION	Build Deliverables.zip from unchanged actual input 1.html, unchanged actual input 2.md, the exact recorded final output 1.html and any actual intermediate artifacts. Open the ZIP and verify its contents are labeled and readable. Do not include this guide, planning docs, metadata or a rehearsal output.

ACTION	Create final Metadata.rtf after recording. Request is the exact approved Goal used in TaskRecorder. Topology lists every actual app visit in chronological order, including returns, using only LOOKUP, CREATE or MODIFY. Side-by-side test copying may produce many alternating Visual Studio Code and Firefox visits; do not replace that real sequence with the shorter proposal outline. Task Description is a numbered high-level account of the work actually done.

ACTION	Upload the untouched matching recorder ZIP, separate matching video.mp4, Deliverables.zip and separate Metadata.rtf in the same approved Studio task. For actual AI-prepared inputs, use the required Final Notes disclosure, AI-generated before recording, truthfully. Run both AutoQCs, resolve or dispute Major findings, and complete only manual checks you actually verified. If you re-record to fix narration, audio or an unexplained error, review and submit the matching new attempt and rerun both AutoQCs; do not reuse old recording artifacts or edit the video. Confirm submission status through Studio; a saved file alone is not submission.